1 Objective

- Design an 8-bit Harvard processor using Verilog HDL.
- Implement instruction fetch, decode, register file, ALU, data memory, and writeback units.
- Execute the given problem: **Fixed Point (Unsigned) Expression Evaluation:** $(a + b + c + d)^2$.
- Validate functionality using simulation test benches.
- Perform logic synthesis using 45nm CMOS technology in Cadence Genus.
- Perform physical design using Cadence Innovus.

2 Problem Statement

The given problem is to compute:

Fixed Point (Unsigned) Expression Evaluation: $(a + b + c + d)^2$

Where:

- a, b, c, d are unsigned 8-bit fixed-point numbers
- The expression involves addition followed by squaring of the result
- All operations are performed using 8-bit fixed-point unsigned arithmetic

array

2.1 Assembly Instructions and 32-bit Binary Words

width=c

Sl No	Instruction	Operation Description	32-bit Binary
1	MOV R1, #1	Load value 1 into R1 (a)	00000000001000000000000000000001
2	MOV R2, #2	Load value 2 into R2 (b)	00000000010000000000000000000010
3	MOV R3, #3	Load value 3 into R3 (c)	00000000011000000000000000000011
4	MOV R4, #4	Load value 4 into R4 (d)	00000000100000000000000000000100
5	ADD R5, R2, R1	R5 ← R2 + R1	00010000000001010000000001000001
6	ADD R5, R5, R3	R5 ← R5 + R3	00010000000001010000000010100011
7	ADD R5, R5, R4	R5 ← R5 + R4	00010000000001010000000010100100
8	MUL R6, R5, R5	R6 ← R5 × R5	00011100111001100000000010100101
9	MOV R7, R6	Move result to R7	00000100111000000000000000000110
10	STORE MEM[8], R6	Store result in memory location 8	00001100001000000000000000000110

Table 1: Assembly Instructions for $(a + b + c + d)^2$ with 32-bit Binary Encoding

/newpage

3 Verilog Modules

3.0.1 Recursive doubling based CLA

```

module four_one_mux(
    input [1:0]i3,i2,i1,i0,
    input [1:0] sel,
    output [1:0] out
);
assign out = (sel== 2'b00)? i0 : (sel == 2'b01)? i1: (sel == 2'b10)? i2 :i3;
endmodule

module four_one_mux_two (
    input  a,
    input  b,
    input  c,
    input  d,
    input  [1:0] sel,
    output out
);

assign out = (sel == 2'b00) ? a :
             (sel == 2'b01) ? b :
             (sel == 2'b10) ? c :
                              d;

endmodule

module cla_8_bit(
    input [7:0]A,B,
    output [7:0] sum,
    output carry_out
);
wire [17:0]kgp;
wire [15:0] kgp1,kgp2,kgp3;
wire [7:0]c;

assign kgp[1:0]=2'b00;
assign kgp[3:2]   = {A[0], B[0]};
assign kgp[5:4]   = {A[1], B[1]};
assign kgp[7:6]   = {A[2], B[2]};
assign kgp[9:8]   = {A[3], B[3]};
assign kgp[11:10] = {A[4], B[4]};
assign kgp[13:12] = {A[5], B[5]};
assign kgp[15:14] = {A[6], B[6]};
assign kgp[17:16] = {A[7], B[7]};

assign kgp1[1:0]= kgp[1:0];

four_one_mux m1 (
    .i0(2'b00),
    .i1(kgp[1:0]),
    .i2(kgp[1:0]),
    .i3(2'b11),
    .sel(kgp[3:2]),
    .out(kgp1[3:2])
);

four_one_mux m2 (
```

```
    .i0(2'b00),
    .i1(kgp[3:2]),
    .i2(kgp[3:2]),
    .i3(2'b11),
    .sel(kgp[5:4]),
    .out(kgp1[5:4])
);

four_one_mux m3 (
    .i0(2'b00),
    .i1(kgp[5:4]),
    .i2(kgp[5:4]),
    .i3(2'b11),
    .sel(kgp[7:6]),
    .out(kgp1[7:6])
);

four_one_mux m4 (.i0(2'b00),.i1(kgp[7:6]),.i2(kgp[7:6]),.i3(2'b11),.sel(kgp[9:8]),.out(kgp1[9:8]));
four_one_mux m5 (.i0(2'b00),.i1(kgp[9:8]),.i2(kgp[9:8]),.i3(2'b11),.sel(kgp[11:10]),.out(kgp1[11:10]));
four_one_mux m6 (.i0(2'b00),.i1(kgp[11:10]),.i2(kgp[11:10]),.i3(2'b11),.sel(kgp[13:12]),.out(kgp1[13:12]));
four_one_mux m7 (.i0(2'b00),.i1(kgp[13:12]),.i2(kgp[13:12]),.i3(2'b11),.sel(kgp[15:14]),.out(kgp1[15:14]));


assign kgp2[1:0]= kgp1[1:0];
assign kgp2[3:2]=kgp1[3:2];

four_one_mux m8 (.i0(2'b00),.i1(kgp1[1:0]),.i2(kgp1[1:0]),.i3(2'b11),.sel(kgp1[5:4]),.out(kgp2[5:4]));
four_one_mux m9 (.i0(2'b00),.i1(kgp1[3:2]),.i2(kgp1[3:2]),.i3(2'b11),.sel(kgp1[7:6]),.out(kgp2[7:6]));
four_one_mux m10 (.i0(2'b00),.i1(kgp1[5:4]),.i2(kgp1[5:4]),.i3(2'b11),.sel(kgp1[9:8]),.out(kgp2[9:8]));
four_one_mux m11 (.i0(2'b00),.i1(kgp1[7:6]),.i2(kgp1[7:6]),.i3(2'b11),.sel(kgp1[11:10]),.out(kgp2[11:10]));
four_one_mux m12 (.i0(2'b00),.i1(kgp1[9:8]),.i2(kgp1[9:8]),.i3(2'b11),.sel(kgp1[13:12]),.out(kgp2[13:12]));
four_one_mux m13 (.i0(2'b00),.i1(kgp1[11:10]),.i2(kgp1[11:10]),.i3(2'b11),.sel(kgp1[15:14]),.out(kgp2[15:14]));



assign kgp3[1:0] = kgp2[1:0];
assign kgp3[3:2] = kgp2[3:2];
assign kgp3[5:4] = kgp2[5:4];
assign kgp3[7:6] = kgp2[7:6];


four_one_mux m14 (.i0(2'b00),.i1(kgp2[1:0]),.i2(kgp2[1:0]),.i3(2'b11),.sel(kgp2[9:8]),.out(kgp3[9:8]));
four_one_mux m15 (.i0(2'b00),.i1(kgp2[3:2]),.i2(kgp2[3:2]),.i3(2'b11),.sel(kgp2[11:10]),.out(kgp3[11:10]));
four_one_mux m16 (.i0(2'b00),.i1(kgp2[5:4]),.i2(kgp2[5:4]),.i3(2'b11),.sel(kgp2[13:12]),.out(kgp3[13:12]));
four_one_mux m17 (.i0(2'b00),.i1(kgp2[7:6]),.i2(kgp2[7:6]),.i3(2'b11),.sel(kgp2[15:14]),.out(kgp3[15:14]));

```

```
assign c[0]= kgp3[1];
assign c[1]= kgp3[3];
assign c[2]= kgp3[5];
assign c[3]= kgp3[7];
assign c[4]= kgp3[9];
assign c[5]= kgp3[11];
assign c[6]= kgp3[13];
assign c[7]= kgp3[15];

four_one_mux_two m111(.a(1'b0),.b(c[7]),.c(c[7]),.d(1'b1),.sel(kgp[17:16]),.
    out(carry_out));

assign sum[0] = A[0] ^ B[0] ^ c[0];
assign sum[1] = A[1] ^ B[1] ^ c[1];
assign sum[2] = A[2] ^ B[2] ^ c[2];
assign sum[3] = A[3] ^ B[3] ^ c[3];
assign sum[4] = A[4] ^ B[4] ^ c[4];
assign sum[5] = A[5] ^ B[5] ^ c[5];
assign sum[6] = A[6] ^ B[6] ^ c[6];
assign sum[7] = A[7] ^ B[7] ^ c[7];

endmodule
```

Listing 1: cla_8bit.v

3.0.2 Subtractor

```

module four_one_mux(
    input  [1:0] i3, i2, i1, i0,
    input  [1:0] sel,
    output [1:0] out
);
assign out = (sel == 2'b00) ? i0 :
             (sel == 2'b01) ? i1 :
             (sel == 2'b10) ? i2 :
                              i3;
endmodule


module four_one_mux_two (
    input  a,
    input  b,
    input  c,
    input  d,
    input  [1:0] sel,
    output out
);
assign out = (sel == 2'b00) ? a :
             (sel == 2'b01) ? b :
             (sel == 2'b10) ? c :
                              d;
endmodule


module sub_8_bit(
    input  [7:0] A, B,
    output [7:0] diff,
    output       borrow_out
```

```
);

    wire [7:0] B_sub;
    wire [17:0] kgp;
    wire [15:0] kgp1, kgp2, kgp3;
    wire [7:0] c;

    // Two's complement inversion
    assign B_sub = ~B;

    // Initial carry-in = 1
    assign kgp[1:0] = 2'b11;

    // KGP generation using A and ~B
    assign kgp[3:2]   = {A[0], B_sub[0]};
    assign kgp[5:4]   = {A[1], B_sub[1]};
    assign kgp[7:6]   = {A[2], B_sub[2]};
    assign kgp[9:8]   = {A[3], B_sub[3]};
    assign kgp[11:10] = {A[4], B_sub[4]};
    assign kgp[13:12] = {A[5], B_sub[5]};
    assign kgp[15:14] = {A[6], B_sub[6]};
    assign kgp[17:16] = {A[7], B_sub[7]};

    // -------- Stage 1 --------
    assign kgp1[1:0] = kgp[1:0];

    four_one_mux m1 (.i0(2'b00), .i1(kgp[1:0]),  .i2(kgp[1:0]),  .i3(2'b11), .sel(kgp[3:2]),   .out(kgp1[3:2]));
    four_one_mux m2 (.i0(2'b00), .i1(kgp[3:2]),  .i2(kgp[3:2]),  .i3(2'b11), .sel(kgp[5:4]),   .out(kgp1[5:4]));
    four_one_mux m3 (.i0(2'b00), .i1(kgp[5:4]),  .i2(kgp[5:4]),  .i3(2'b11), .sel(kgp[7:6]),   .out(kgp1[7:6]));
    four_one_mux m4 (.i0(2'b00), .i1(kgp[7:6]),  .i2(kgp[7:6]),  .i3(2'b11), .sel(kgp[9:8]),   .out(kgp1[9:8]));
    four_one_mux m5 (.i0(2'b00), .i1(kgp[9:8]),  .i2(kgp[9:8]),  .i3(2'b11), .sel(kgp[11:10]), .out(kgp1[11:10]));
    four_one_mux m6 (.i0(2'b00), .i1(kgp[11:10]),.i2(kgp[11:10]),.i3(2'b11), .sel(kgp[13:12]), .out(kgp1[13:12]));
    four_one_mux m7 (.i0(2'b00), .i1(kgp[13:12]),.i2(kgp[13:12]),.i3(2'b11), .sel(kgp[15:14]), .out(kgp1[15:14]));

    // -------- Stage 2 --------
    assign kgp2[1:0] = kgp1[1:0];
    assign kgp2[3:2] = kgp1[3:2];

    four_one_mux m8  (.i0(2'b00), .i1(kgp1[1:0]),  .i2(kgp1[1:0]),  .i3(2'b11), .sel(kgp1[5:4]),   .out(kgp2[5:4]));
    four_one_mux m9  (.i0(2'b00), .i1(kgp1[3:2]),  .i2(kgp1[3:2]),  .i3(2'b11), .sel(kgp1[7:6]),   .out(kgp2[7:6]));
    four_one_mux m10 (.i0(2'b00), .i1(kgp1[5:4]),  .i2(kgp1[5:4]),  .i3(2'b11), .sel(kgp1[9:8]),   .out(kgp2[9:8]));
    four_one_mux m11 (.i0(2'b00), .i1(kgp1[7:6]),  .i2(kgp1[7:6]),  .i3(2'b11), .sel(kgp1[11:10]), .out(kgp2[11:10]));
    four_one_mux m12 (.i0(2'b00), .i1(kgp1[9:8]),  .i2(kgp1[9:8]),  .i3(2'b11), .sel(kgp1[13:12]), .out(kgp2[13:12]));
    four_one_mux m13 (.i0(2'b00), .i1(kgp1[11:10]),.i2(kgp1[11:10]),.i3(2'b11), .sel(kgp1[15:14]), .out(kgp2[15:14]));

    // -------- Stage 3 --------
    assign kgp3[1:0] = kgp2[1:0];
```

```
    assign kgp3[3:2] = kgp2[3:2];
    assign kgp3[5:4] = kgp2[5:4];
    assign kgp3[7:6] = kgp2[7:6];

    four_one_mux m14 (.i0(2'b00), .i1(kgp2[1:0]),  .i2(kgp2[1:0]),  .i3(2'b11), .sel(kgp2[9:8]),   .out(kgp3[9:8]));
    four_one_mux m15 (.i0(2'b00), .i1(kgp2[3:2]),  .i2(kgp2[3:2]),  .i3(2'b11), .sel(kgp2[11:10]), .out(kgp3[11:10]));
    four_one_mux m16 (.i0(2'b00), .i1(kgp2[5:4]),  .i2(kgp2[5:4]),  .i3(2'b11), .sel(kgp2[13:12]), .out(kgp3[13:12]));
    four_one_mux m17 (.i0(2'b00), .i1(kgp2[7:6]),  .i2(kgp2[7:6]),  .i3(2'b11), .sel(kgp2[15:14]), .out(kgp3[15:14]));

    // Carry extraction
    assign c[0] = kgp3[1];
    assign c[1] = kgp3[3];
    assign c[2] = kgp3[5];
    assign c[3] = kgp3[7];
    assign c[4] = kgp3[9];
    assign c[5] = kgp3[11];
    assign c[6] = kgp3[13];
    assign c[7] = kgp3[15];

    // Final borrow logic
    four_one_mux_two m111 (
        .a(1'b0), .b(c[7]), .c(c[7]), .d(1'b1),
        .sel(kgp[17:16]),
        .out(borrow_out)
    );

    // Difference calculation
    assign diff[0] = A[0] ^ B_sub[0] ^ c[0];
    assign diff[1] = A[1] ^ B_sub[1] ^ c[1];
    assign diff[2] = A[2] ^ B_sub[2] ^ c[2];
    assign diff[3] = A[3] ^ B_sub[3] ^ c[3];
    assign diff[4] = A[4] ^ B_sub[4] ^ c[4];
    assign diff[5] = A[5] ^ B_sub[5] ^ c[5];
    assign diff[6] = A[6] ^ B_sub[6] ^ c[6];
    assign diff[7] = A[7] ^ B_sub[7] ^ c[7];

endmodule
```

Listing 2: sub_8bit.v

3.0.3 Wallace Tree Multiplier

```
module full_adder(
    input a,b,c,
    output sum,carry
);
assign sum = a^b^c;
assign carry = (a&b) | (b&c) | (c&a);
endmodule

module half_adder(
    input x,y,
    output sum,carry
);
assign sum= x^y;
```

```
assign carry= x&y;
endmodule



module cla_16bit (
    input  [15:0] a,
    input  [15:0] b,
    input         cin,
    output [15:0] sum,
    output        cout
);

    wire [15:0] p, g;
    wire [16:0] c;

    assign p = a ^ b;
    assign g = a & b;

    assign c[0] = cin;

    assign c[1]  = g[0]  | (p[0]  & c[0]);
    assign c[2]  = g[1]  | (p[1]  & g[0]) |
                   (p[1]  & p[0]  & c[0]);

    assign c[3]  = g[2]  | (p[2]  & g[1]) |
                   (p[2]  & p[1]  & g[0]) |
                   (p[2]  & p[1]  & p[0]  & c[0]);

    assign c[4]  = g[3]  | (p[3]  & g[2]) |
                   (p[3]  & p[2]  & g[1]) |
                   (p[3]  & p[2]  & p[1]  & g[0]) |
                   (p[3]  & p[2]  & p[1]  & p[0]  & c[0]);

    assign c[5]  = g[4]  | (p[4]  & c[4]);
    assign c[6]  = g[5]  | (p[5]  & g[4]) |
                   (p[5]  & p[4]  & c[4]);

    assign c[7]  = g[6]  | (p[6]  & g[5]) |
                   (p[6]  & p[5]  & g[4]) |
                   (p[6]  & p[5]  & p[4]  & c[4]);

    assign c[8]  = g[7]  | (p[7]  & g[6]) |
                   (p[7]  & p[6]  & g[5]) |
                   (p[7]  & p[6]  & p[5]  & g[4]) |
                   (p[7]  & p[6]  & p[5]  & p[4]  & c[4]);

    assign c[9]  = g[8]  | (p[8]  & c[8]);
    assign c[10] = g[9]  | (p[9]  & g[8]) |
                   (p[9]  & p[8]  & c[8]);

    assign c[11] = g[10] | (p[10] & g[9]) |
                   (p[10] & p[9]  & g[8]) |
                   (p[10] & p[9]  & p[8]  & c[8]);

    assign c[12] = g[11] | (p[11] & g[10]) |
                   (p[11] & p[10] & g[9]) |
                   (p[11] & p[10] & p[9]  & g[8]) |
                   (p[11] & p[10] & p[9]  & p[8]  & c[8]);

```

```
    assign c[13] = g[12] | (p[12] & c[12]);
    assign c[14] = g[13] | (p[13] & g[12]) |
                   (p[13] & p[12] & c[12]);

    assign c[15] = g[14] | (p[14] & g[13]) |
                   (p[14] & p[13] & g[12]) |
                   (p[14] & p[13] & p[12] & c[12]);

    assign c[16] = g[15] | (p[15] & g[14]) |
                   (p[15] & p[14] & g[13]) |
                   (p[15] & p[14] & p[13] & g[12]) |
                   (p[15] & p[14] & p[13] & p[12] & c[12]);

    assign sum  = p ^ c[15:0];
    assign cout = c[16];

endmodule

module wallace_tree_multiplier(
     input  [7:0]  a,
    input  [7:0]  b,
    output [15:0] mul_out
);


wire [7:0] p_p1, p_p2, p_p3, p_p4;
    wire [7:0] p_p5, p_p6, p_p7, p_p8;

assign p_p1[0] = a[0] & b[0];
assign p_p1[1] = a[1] & b[0];
assign p_p1[2] = a[2] & b[0];
assign p_p1[3] = a[3] & b[0];
assign p_p1[4] = a[4] & b[0];
assign p_p1[5] = a[5] & b[0];
assign p_p1[6] = a[6] & b[0];
assign p_p1[7] = a[7] & b[0];


assign p_p2[0] = a[0] & b[1];
assign p_p2[1] = a[1] & b[1];
assign p_p2[2] = a[2] & b[1];
assign p_p2[3] = a[3] & b[1];
assign p_p2[4] = a[4] & b[1];
assign p_p2[5] = a[5] & b[1];
assign p_p2[6] = a[6] & b[1];
assign p_p2[7] = a[7] & b[1];


assign p_p3[0] = a[0] & b[2];
assign p_p3[1] = a[1] & b[2];
assign p_p3[2] = a[2] & b[2];
assign p_p3[3] = a[3] & b[2];
assign p_p3[4] = a[4] & b[2];
assign p_p3[5] = a[5] & b[2];
assign p_p3[6] = a[6] & b[2];
assign p_p3[7] = a[7] & b[2];


assign p_p4[0] = a[0] & b[3];
assign p_p4[1] = a[1] & b[3];
```

```
assign p_p4[2] = a[2] & b[3];
assign p_p4[3] = a[3] & b[3];
assign p_p4[4] = a[4] & b[3];
assign p_p4[5] = a[5] & b[3];
assign p_p4[6] = a[6] & b[3];
assign p_p4[7] = a[7] & b[3];


assign p_p5[0] = a[0] & b[4];
assign p_p5[1] = a[1] & b[4];
assign p_p5[2] = a[2] & b[4];
assign p_p5[3] = a[3] & b[4];
assign p_p5[4] = a[4] & b[4];
assign p_p5[5] = a[5] & b[4];
assign p_p5[6] = a[6] & b[4];
assign p_p5[7] = a[7] & b[4];


assign p_p6[0] = a[0] & b[5];
assign p_p6[1] = a[1] & b[5];
assign p_p6[2] = a[2] & b[5];
assign p_p6[3] = a[3] & b[5];
assign p_p6[4] = a[4] & b[5];
assign p_p6[5] = a[5] & b[5];
assign p_p6[6] = a[6] & b[5];
assign p_p6[7] = a[7] & b[5];


assign p_p7[0] = a[0] & b[6];
assign p_p7[1] = a[1] & b[6];
assign p_p7[2] = a[2] & b[6];
assign p_p7[3] = a[3] & b[6];
assign p_p7[4] = a[4] & b[6];
assign p_p7[5] = a[5] & b[6];
assign p_p7[6] = a[6] & b[6];
assign p_p7[7] = a[7] & b[6];


assign p_p8[0] = a[0] & b[7];
assign p_p8[1] = a[1] & b[7];
assign p_p8[2] = a[2] & b[7];
assign p_p8[3] = a[3] & b[7];
assign p_p8[4] = a[4] & b[7];
assign p_p8[5] = a[5] & b[7];
assign p_p8[6] = a[6] & b[7];
assign p_p8[7] = a[7] & b[7];

//csa-1

wire [9:0] sum_1;
wire [7:0] carry_1;
assign sum_1[0]=p_p1[0];
half_adder ha0(.x(p_p1[1]),.y(p_p2[0]),.sum(sum_1[1]),.carry(carry_1[0]));

full_adder fa0(.a(p_p1[2]), .b(p_p2[1]), .c(p_p3[0]), .sum(sum_1[2]), .carry
    (carry_1[1]));
full_adder fa1(.a(p_p1[3]), .b(p_p2[2]), .c(p_p3[1]), .sum(sum_1[3]), .carry
    (carry_1[2]));
full_adder fa2(.a(p_p1[4]), .b(p_p2[3]), .c(p_p3[2]), .sum(sum_1[4]), .carry
    (carry_1[3]));
```

```
full_adder fa3(.a(p_p1[5]), .b(p_p2[4]), .c(p_p3[3]), .sum(sum_1[5]), .carry(carry_1[4]));
full_adder fa4(.a(p_p1[6]), .b(p_p2[5]), .c(p_p3[4]), .sum(sum_1[6]), .carry(carry_1[5]));
full_adder fa5(.a(p_p1[7]), .b(p_p2[6]), .c(p_p3[5]), .sum(sum_1[7]), .carry(carry_1[6]));

half_adder ha1(.x(p_p2[7]),.y(p_p3[6]),.sum(sum_1[8]),.carry(carry_1[7]));
assign sum_1[9]= p_p3[7];

//csa_2
wire [9:0] sum_2;
wire [7:0] carry_2;

assign sum_2[0]=p_p4[0];
half_adder ha2(.x(p_p4[1]),.y(p_p5[0]),.sum(sum_2[1]),.carry(carry_2[0]));

full_adder fa6(.a(p_p4[2]), .b(p_p5[1]), .c(p_p6[0]), .sum(sum_2[2]), .carry(carry_2[1]));
full_adder fa7(.a(p_p4[3]), .b(p_p5[2]), .c(p_p6[1]), .sum(sum_2[3]), .carry(carry_2[2]));
full_adder fa8(.a(p_p4[4]), .b(p_p5[3]), .c(p_p6[2]), .sum(sum_2[4]), .carry(carry_2[3]));
full_adder fa9(.a(p_p4[5]), .b(p_p5[4]), .c(p_p6[3]), .sum(sum_2[5]), .carry(carry_2[4]));
full_adder fa10(.a(p_p4[6]), .b(p_p5[5]), .c(p_p6[4]), .sum(sum_2[6]), .carry(carry_2[5]));
full_adder fa11(.a(p_p4[7]), .b(p_p5[6]), .c(p_p6[5]), .sum(sum_2[7]), .carry(carry_2[6]));

half_adder ha3(.x(p_p5[7]),.y(p_p6[6]),.sum(sum_2[8]),.carry(carry_2[7]));
assign sum_2[9]= p_p6[7];
//csa_3
wire [12:0] sum_3;
wire [7:0] carry_3;

assign sum_3[0]=sum_1[0];
assign sum_3[1] = sum_1[1];

half_adder ha4(.x(sum_1[2]),.y(carry_1[0]),.sum(sum_3[2]),.carry(carry_3[0]));

full_adder fa12(.a(sum_1[3]), .b(carry_1[1]), .c(sum_2[0]), .sum(sum_3[3]), .carry(carry_3[1]));
full_adder fa13(.a(sum_1[4]), .b(carry_1[2]), .c(sum_2[1]), .sum(sum_3[4]), .carry(carry_3[2]));
full_adder fa14(.a(sum_1[5]), .b(carry_1[3]), .c(sum_2[2]), .sum(sum_3[5]), .carry(carry_3[3]));
full_adder fa15(.a(sum_1[6]), .b(carry_1[4]), .c(sum_2[3]), .sum(sum_3[6]), .carry(carry_3[4]));
full_adder fa16(.a(sum_1[7]), .b(carry_1[5]), .c(sum_2[4]), .sum(sum_3[7]), .carry(carry_3[5]));
full_adder fa17(.a(sum_1[8]), .b(carry_1[6]), .c(sum_2[5]), .sum(sum_3[8]), .carry(carry_3[6]));
full_adder fa18(.a(sum_1[9]), .b(carry_1[7]), .c(sum_2[6]), .sum(sum_3[9]), .carry(carry_3[7]));

assign sum_3[10]= sum_2[7];
assign sum_3[11]= sum_2[8];
assign sum_3[12]= sum_2[9];
```

```
//csa_4

wire [9:0] sum_4;
wire [7:0] carry_4;

assign sum_4[0] = carry_2[0];

half_adder ha5(.x(p_p7[0]),.y(carry_2[1]),.sum(sum_4[1]),.carry(carry_4[0]));

full_adder fa19(.a(p_p7[1]), .b(p_p8[0]), .c(carry_2[2]), .sum(sum_4[2]), .carry(carry_4[1]));
full_adder fa20(.a(p_p7[2]), .b(p_p8[1]), .c(carry_2[3]), .sum(sum_4[3]), .carry(carry_4[2]));
full_adder fa21(.a(p_p7[3]), .b(p_p8[2]), .c(carry_2[4]), .sum(sum_4[4]), .carry(carry_4[3]));
full_adder fa22(.a(p_p7[4]), .b(p_p8[3]), .c(carry_2[5]), .sum(sum_4[5]), .carry(carry_4[4]));
full_adder fa23(.a(p_p7[5]), .b(p_p8[4]), .c(carry_2[6]), .sum(sum_4[6]), .carry(carry_4[5]));
full_adder fa24(.a(p_p7[6]), .b(p_p8[5]), .c(carry_2[7]), .sum(sum_4[7]), .carry(carry_4[6]));

half_adder ha6(.x(p_p7[7]),.y(p_p8[6]),.sum(sum_4[8]),.carry(carry_4[7]));
assign sum_4[9] =p_p8[7];

//csa_5
wire [14:0] sum_5;
wire [9:0] carry_5;

assign sum_5[0]= sum_3[0];
assign sum_5[1]= sum_3[1];
assign sum_5[2]= sum_3[2];

half_adder ha7(.x(sum_3[3]),.y(carry_3[0]),.sum(sum_5[3]),.carry(carry_5[0]));
half_adder ha8(.x(sum_3[4]),.y(carry_3[1]),.sum(sum_5[4]),.carry(carry_5[1]));

full_adder fa25(.a(sum_3[5]),  .b(carry_3[2]), .c(sum_4[0]), .sum(sum_5[5]), .carry(carry_5[2]));
full_adder fa26(.a(sum_3[6]),  .b(carry_3[3]), .c(sum_4[1]), .sum(sum_5[6]), .carry(carry_5[3]));
full_adder fa27(.a(sum_3[7]),  .b(carry_3[4]), .c(sum_4[2]), .sum(sum_5[7]), .carry(carry_5[4]));
full_adder fa28(.a(sum_3[8]),  .b(carry_3[5]), .c(sum_4[3]), .sum(sum_5[8]), .carry(carry_5[5]));
full_adder fa29(.a(sum_3[9]),  .b(carry_3[6]), .c(sum_4[4]), .sum(sum_5[9]), .carry(carry_5[6]));
full_adder fa30(.a(sum_3[10]), .b(carry_3[7]), .c(sum_4[5]), .sum(sum_5[10]), .carry(carry_5[7]));


half_adder ha9(.x(sum_3[11]),.y(sum_4[6]),.sum(sum_5[11]),.carry(carry_5[8]));
half_adder ha10(.x(sum_3[12]),.y(sum_4[7]),.sum(sum_5[12]),.carry(carry_5[9]));

assign sum_5[13]=sum_4[8];
```

```
assign sum_5[14]=sum_4[9];

//csa_6

wire [14:0] sum_6;
wire [10:0] carry_6;
assign sum_6[0]=sum_5[0];
assign sum_6[1]=sum_5[1];
assign sum_6[2]=sum_5[2];
assign sum_6[3]=sum_5[3];

half_adder ha11(.x(sum_5[4]), .y(carry_5[0]), .sum(sum_6[4]), .carry(carry_6[0]));
half_adder ha12(.x(sum_5[5]), .y(carry_5[1]), .sum(sum_6[5]), .carry(carry_6[1]));
half_adder ha13(.x(sum_5[6]), .y(carry_5[2]), .sum(sum_6[6]), .carry(carry_6[2]));

full_adder fa31(.a(sum_5[7]),  .b(carry_5[3]), .c(carry_4[0]), .sum(sum_6[7]),  .carry(carry_6[3]));
full_adder fa32(.a(sum_5[8]),  .b(carry_5[4]), .c(carry_4[1]), .sum(sum_6[8]),  .carry(carry_6[4]));
full_adder fa33(.a(sum_5[9]),  .b(carry_5[5]), .c(carry_4[2]), .sum(sum_6[9]),  .carry(carry_6[5]));
full_adder fa34(.a(sum_5[10]), .b(carry_5[6]), .c(carry_4[3]), .sum(sum_6[10]), .carry(carry_6[6]));
full_adder fa35(.a(sum_5[11]), .b(carry_5[7]), .c(carry_4[4]), .sum(sum_6[11]), .carry(carry_6[7]));
full_adder fa36(.a(sum_5[12]), .b(carry_5[8]), .c(carry_4[5]), .sum(sum_6[12]), .carry(carry_6[8]));
full_adder fa37(.a(sum_5[13]), .b(carry_5[9]), .c(carry_4[6]), .sum(sum_6[13]), .carry(carry_6[9]));

half_adder ha14(.x(sum_5[14]), .y(carry_4[7]), .sum(sum_6[14]), .carry(carry_6[10]));

wire [15:0] final_sum;
wire [15:0] final_carry;

assign final_sum   = {1'b0, sum_6};
assign final_carry = {carry_6, 5'b00000};

cla_16bit c1(
    .a(final_sum),
    .b(final_carry),
    .cin(1'b0),
    .sum(mul_out),
    .cout()
);

endmodule
```

Listing 3: wallace_tree_multiplier.v

3.0.4 Non-Restoring Divider

```

module mux_2_1(
    input [7:0] x,y,
```

```
    input  sel,
    output [7:0] out_mux
);

assign out_mux= (sel==1) ? y : x;
endmodule


module stage(
    input [7:0] x,//accumulator
    input [7:0] Q,// dividend
    input [7:0] M,//divsor
    output   [7:0] Z,
    output  reg [7:0] Q2
);
reg [7:0] M1;
wire [7:0] Q1;

wire  [7:0] x1;



wire [7:0] MR;
assign MR = (~M) + 8'd1;

assign x1 = {x[6:0], Q[7]};
assign Q1 = {Q[6:0], 1'b0};

always@(*) begin
    if(x[7]) begin

        M1[7:0]=M[7:0];
    end
    else begin

        M1[7:0]=MR[7:0];
    end
end

cla_8_bit c1(.A(x1),.B(M1),.sum(Z));

always@(*) begin
    if(Z[7]) begin
        Q2[7:0]={Q1[7:1],1'b0};
    end
    else begin
        Q2[7:0] ={Q1[7:1],1'b1};
    end
end
endmodule


module non_rst_div(
    input [6:0] divisor,
    input [7:0] dividend,
    output [7:0] quotient,
    output [7:0] remainder
);

wire  [7:0] AA;
```

```
assign AA=8'd0;



wire [7:0] M = {1'b0, divisor};

//stage_0
wire [7:0] AA0,QQ0;

stage stage_0(.Q(dividend),.M(M),.x(AA),.Z(AA0),.Q2(QQ0));

//satge_1
wire [7:0] AA1,QQ1;

stage stage_1(.Q(QQ0),.M(M),.x(AA0),.Z(AA1),.Q2(QQ1));

//stage_2
wire [7:0] AA2,QQ2;

stage stage_2(.Q(QQ1),.M(M),.x(AA1),.Z(AA2),.Q2(QQ2));

//stage_3
wire [7:0] AA3,QQ3;

stage stage_3(.Q(QQ2),.M(M),.x(AA2),.Z(AA3),.Q2(QQ3));

//stage_4

wire [7:0] AA4,QQ4;

stage stage_4(.Q(QQ3),.M(M),.x(AA3),.Z(AA4),.Q2(QQ4));

//satge_5

wire [7:0] AA5,QQ5;

stage stage_5(.Q(QQ4),.M(M),.x(AA4),.Z(AA5),.Q2(QQ5));


//stage_6

wire [7:0] AA6,QQ6;

stage stage_6(.Q(QQ5),.M(M),.x(AA5),.Z(AA6),.Q2(QQ6));


//stage_7
wire [7:0] AA7,QQ7;

stage stage_7(.Q(QQ6),.M(M),.x(AA6),.Z(AA7),.Q2(QQ7));

wire [7:0] remainder1;
wire [7:0] sum_out;
cla_8_bit c2(.A(AA7),.B(M),.sum(sum_out),.carry_out());


```

```
mux_2_1 m1(.x(AA7),.y(sum_out),.out_mux(remainder1),.sel(AA7[7]));

assign quotient  = (divisor == 0) ? 8'd0 : QQ7;
assign remainder = (divisor == 0) ? dividend : remainder1;

endmodule
```

Listing 4: non_restoring_division.v

3.0.5 Logical Operations

AND Operation

```
module eight_bit_and(
    input [7:0] A,
    input [7:0] B,
    output [7:0] out
);

assign out=A&B;
endmodule
```

Listing 5: eight_bit_and.v

OR Operation

```
module eight_bit_or(
      input [7:0] A,
    input [7:0] B,
    output [7:0] out
);
assign out= A|B;
endmodule
```

Listing 6: eight_bit_or.v

XOR Operation

```
module eight_bit_xor(
      input [7:0] A,
    input [7:0] B,
    output [7:0] out
);
assign out=(A^B);
endmodule
```

Listing 7: eight_bit_xor.v

NAND Operation

```
module eight_bit_nand(
      input [7:0] A,
    input [7:0] B,
    output [7:0] out
);
assign out=~(A&B);
endmodule
```

Listing 8: eight_bit_nand.v

NOR Operation

```
module eight_bit_nor(
      input [7:0] A,
    input [7:0] B,
    output [7:0] out
);
assign out=~(A|B);
endmodule
```

Listing 9: `eight_bit_nor.v`

XNOR Operation

```
module eight_bit_xnor(
      input [7:0] A,
    input [7:0] B,
    output [7:0] out
);

assign out= ~(A^B);
endmodule
```

Listing 10: `eight_bit_xnor.v`

NOT Operation

```

module eight_bit_not(
    input [7:0] A,
    output [7:0] out
);

assign  out=~(A);
endmodule
```

Listing 11: `eight_bit_not.v`

3.0.6 Shift Operations

Logical Left Shift

```

module two_one_mux(
    input a,b,
    input sel,
    output out
);

assign out = (sel==1)? a:b;
endmodule

module eight_bit_log_left(
    input [7:0] in,
    output [7:0] shift_out,
    input [2:0] sel
);


//stage_1
wire [7:0] w;


```

```
two_one_mux m0(.a(1'b0),.b(in[0]),.out(w[0]),.sel(sel[0]));
two_one_mux m1(.a(in[0]),.b(in[1]),.out(w[1]),.sel(sel[0]));
two_one_mux m2(.a(in[1]),.b(in[2]),.out(w[2]),.sel(sel[0]));
two_one_mux m3(.a(in[2]),.b(in[3]),.out(w[3]),.sel(sel[0]));
two_one_mux m4(.a(in[3]),.b(in[4]),.out(w[4]),.sel(sel[0]));
two_one_mux m5(.a(in[4]),.b(in[5]),.out(w[5]),.sel(sel[0]));
two_one_mux m6(.a(in[5]),.b(in[6]),.out(w[6]),.sel(sel[0]));
two_one_mux m7(.a(in[6]),.b(in[7]),.out(w[7]),.sel(sel[0]));

//stage_2
wire [7:0] w1;


two_one_mux m8(.a(1'b0),.b(w[0]),.sel(sel[1]),.out(w1[0]));
two_one_mux m9(.a(1'b0),.b(w[1]),.sel(sel[1]),.out(w1[1]));
two_one_mux m10(.a(w[0]),.b(w[2]),.sel(sel[1]),.out(w1[2]));
two_one_mux m11(.a(w[1]),.b(w[3]),.sel(sel[1]),.out(w1[3]));
two_one_mux m12(.a(w[2]),.b(w[4]),.sel(sel[1]),.out(w1[4]));
two_one_mux m13(.a(w[3]),.b(w[5]),.sel(sel[1]),.out(w1[5]));
two_one_mux m14(.a(w[4]),.b(w[6]),.sel(sel[1]),.out(w1[6]));
two_one_mux m15(.a(w[5]),.b(w[7]),.sel(sel[1]),.out(w1[7]));


//stage_3



two_one_mux m16(.a(1'b0),.b(w1[0]),.sel(sel[2]),.out(shift_out[0]));
two_one_mux m17(.a(1'b0),.b(w1[1]),.sel(sel[2]),.out(shift_out[1]));
two_one_mux m18(.a(1'b0),.b(w1[2]),.sel(sel[2]),.out(shift_out[2]));
two_one_mux m19(.a(1'b0),.b(w1[3]),.sel(sel[2]),.out(shift_out[3]));
two_one_mux m20(.a(w1[0]),.b(w1[4]),.sel(sel[2]),.out(shift_out[4]));
two_one_mux m21(.a(w1[1]),.b(w1[5]),.sel(sel[2]),.out(shift_out[5]));
two_one_mux m22(.a(w1[2]),.b(w1[6]),.sel(sel[2]),.out(shift_out[6]));
two_one_mux m23(.a(w1[3]),.b(w1[7]),.sel(sel[2]),.out(shift_out[7]));

endmodule
```

Listing 12: eight_bit_log_left.v

Logical Right Shift

```
module two_one_mux(
    input a,b,
    input sel,
    output out
);

assign out = (sel==1)? a:b;
endmodule


module eight_bit_log_right(
    input [7:0] in,
    output [7:0] shift_out,
    input [2:0] sel
);
 //stage_1

 wire [7:0] w;

```

```
two_one_mux m0(.a(in[1]), .b(in[0]), .sel(sel[0]), .out(w[0]));
two_one_mux m1(.a(in[2]), .b(in[1]), .sel(sel[0]), .out(w[1]));
two_one_mux m2(.a(in[3]), .b(in[2]), .sel(sel[0]), .out(w[2]));
two_one_mux m3(.a(in[4]), .b(in[3]), .sel(sel[0]), .out(w[3]));
two_one_mux m4(.a(in[5]), .b(in[4]), .sel(sel[0]), .out(w[4]));
two_one_mux m5(.a(in[6]), .b(in[5]), .sel(sel[0]), .out(w[5]));
two_one_mux m6(.a(in[7]), .b(in[6]), .sel(sel[0]), .out(w[6]));
two_one_mux m7(.a(1'b0), .b(in[7]), .sel(sel[0]), .out(w[7]));


//stage_2

wire [7:0] w1;

two_one_mux m8(.a(w[2]),.b(w[0]),.sel(sel[1]),.out(w1[0]));
two_one_mux m9(.a(w[3]),.b(w[1]),.sel(sel[1]),.out(w1[1]));
two_one_mux m10(.a(w[4]),.b(w[2]),.sel(sel[1]),.out(w1[2]));
two_one_mux m11(.a(w[5]),.b(w[3]),.sel(sel[1]),.out(w1[3]));
two_one_mux m12(.a(w[6]),.b(w[4]),.sel(sel[1]),.out(w1[4]));
two_one_mux m13(.a(w[7]),.b(w[5]),.sel(sel[1]),.out(w1[5]));
two_one_mux m14(.a(1'b0),.b(w[6]),.sel(sel[1]),.out(w1[6]));
two_one_mux m15(.a(1'b0),.b(w[7]),.sel(sel[1]),.out(w1[7]));

//stage_3

two_one_mux m16(.a(w1[4]),.b(w1[0]),.sel(sel[2]),.out(shift_out[0]));
two_one_mux m17(.a(w1[5]),.b(w1[1]),.sel(sel[2]),.out(shift_out[1]));
two_one_mux m18(.a(w1[6]),.b(w1[2]),.sel(sel[2]),.out(shift_out[2]));
two_one_mux m19(.a(w1[7]),.b(w1[3]),.sel(sel[2]),.out(shift_out[3]));
two_one_mux m20(.a(1'b0),.b(w1[4]),.sel(sel[2]),.out(shift_out[4]));
two_one_mux m21(.a(1'b0),.b(w1[5]),.sel(sel[2]),.out(shift_out[5]));
two_one_mux m22(.a(1'b0),.b(w1[6]),.sel(sel[2]),.out(shift_out[6]));
two_one_mux m23(.a(1'b0),.b(w1[7]),.sel(sel[2]),.out(shift_out[7]));

endmodule
```

Listing 13: eight_bit_log_right.v

3.1 Fetch Unit

```


module fetch_unit(
    input clk,
    input reset,
    input write_en,
    input  [5:0]  read_address,
    output [31:0] instruction_out,
    input  [31:0] instruction_in
);

    reg [31:0] inst_mem [0:63];
    reg [5:0]  counter;

    always @(posedge clk) begin
        if (reset)
            counter <= 6'd0;
        else if (write_en) begin
            inst_mem[counter] <= instruction_in;
```

```
            counter <= counter + 1'b1;
        end
    end

    assign instruction_out = inst_mem[read_address];

endmodule
```

Listing 14: fetch_unit.v

3.2 Register File

```



module register_file(
    input clk,
    input write_enable,
    input  [4:0] read_address,
    input  [4:0] write_address,
    input  [7:0] write_data,
    output [7:0] data_out
);
    reg [7:0] registers[31:0];


    always @(posedge clk) begin
        if (write_enable)
            registers[write_address] <= write_data;
    end

    assign data_out = registers[read_address];

endmodule
```

Listing 15: register_file.v

3.3 Decode Unit

```

module decode_unit(
    input  [31:0] instruction,
    output reg [4:0] read_address1,
    output reg [4:0] read_address2
);



wire [5:0] opcode;

assign opcode = instruction[31:26];


always @(*) begin

    read_address1 = 5'd0;
    read_address2 = 5'd0;
// 000011 STORE
```

```
  if (opcode == 6'b000011) begin
    read_address1 = instruction[4:0];  // must read R6
    read_address2 = 5'd0;
   end

//000001 REGISTER_MOVEE
else if(opcode==6'd1) begin
    read_address1=instruction[4:0];
end


    // 000100 ADD
   else  if (opcode == 6'b000100) begin
        read_address1 = instruction[4:0];
        read_address2 = instruction[9:5];
    end

    // 000101 SUB
    else if (opcode == 6'b000101) begin
        read_address1 = instruction[4:0];
        read_address2 = instruction[9:5];
    end

    // 000110 NEG
    else if (opcode == 6'b000110) begin
        read_address1 = instruction[4:0];
    end

    // 000111 MUL
    else if (opcode == 6'b000111) begin
        read_address1 = instruction[4:0];
        read_address2 = instruction[9:5];
    end

    // 001000 DIV
    else if (opcode == 6'b001000) begin
        read_address1 = instruction[4:0];
        read_address2 = instruction[9:5];
    end

    // 001001 OR
    else if (opcode == 6'b001001) begin
        read_address1 = instruction[4:0];
        read_address2 = instruction[9:5];
    end

    // 001010 XOR
    else if (opcode == 6'b001010) begin
        read_address1 = instruction[4:0];
        read_address2 = instruction[9:5];
    end

    // 001011 NAND
    else if (opcode == 6'b001011) begin
        read_address1 = instruction[4:0];
        read_address2 = instruction[9:5];
    end

    // 001100 NOR
    else if (opcode == 6'b001100) begin
```

```
        read_address1 = instruction[4:0];
        read_address2 = instruction[9:5];
    end

    // 001101 XNOR
    else if (opcode == 6'b001101) begin
        read_address1 = instruction[4:0];
        read_address2 = instruction[9:5];
    end

    // 001110 NOT
    else if (opcode == 6'b001110) begin
        read_address1 = instruction[4:0];
    end

    // 001111 LSHL
    else if (opcode == 6'b001111) begin
        read_address1 = instruction[4:0];
        read_address2 = instruction[9:5];
    end

    // 010000 LRSH
    else if (opcode == 6'b010000) begin
        read_address1 = instruction[4:0];
        read_address2 = instruction[9:5];
    end

end

endmodule
```

Listing 16: decode_unit.v

3.4 ALU

```
`include "cla_8_bit.v"
`include "sub_8_bit.v"
`include "wallace_tree_multiplier.v"
`include "non_rst_div.v"
`include "eight_bit_neg.v"
`include "eight_bit_log_left.v"
`include "eight_bit_log_right.v"
`include "eight_bit_or.v"
`include "eight_bit_and.v"
`include "eight_bit_nand.v"
`include "eight_bit_xor.v"
`include "eight_bit_xnor.v"
`include "eight_bit_not.v"
`include "eight_bit_nor.v"

module alu(
    input  [5:0] opcode,
    input  [7:0] A,
    input  [7:0] B,
    output reg [7:0] output1,
    output reg [7:0] output2
);

// Intermediate wires
```

```
wire [7:0] add_sum;
wire       add_carry;

wire [7:0] sub_diff;
wire       sub_borrow;

wire [15:0] mul_full;

wire [7:0] div_quot;
wire [7:0] div_rem;

wire [7:0] neg_out;
wire [7:0] lshift_out;
wire [7:0] rshift_out;

wire [7:0] or_out;
wire [7:0] and_out;
wire [7:0] nand_out;
wire [7:0] xor_out;
wire [7:0] xnor_out;
wire [7:0] not_out;
wire [7:0] nor_out;

// Instantiate modules

cla_8_bit sum(
    .A(A),
    .B(B),
    .sum(add_sum),
    .carry_out(add_carry)
);

sub_8_bit sub(
    .A(A),
    .B(B),
    .diff(sub_diff),
    .borrow_out(sub_borrow)
);

wallace_tree_multiplier mul(
    .a(A),
    .b(B),
    .mul_out(mul_full)
);

non_rst_div div(
    .dividend(A),
    .divisor(B[6:0]),
    .quotient(div_quot),
    .remainder(div_rem)
);

eight_bit_neg neg(.A(A), .out(neg_out));
eight_bit_log_left left_shift(.in(A), .sel(B[2:0]), .shift_out(lshift_out));
eight_bit_log_right right_shift(.in(A), .sel(B[2:0]), .shift_out(rshift_out)
    );

eight_bit_or or_gate(.A(A), .B(B), .out(or_out));
eight_bit_and and_gate(.A(A), .B(B), .out(and_out));
eight_bit_nand nand_gate(.A(A), .B(B), .out(nand_out));
```

```
eight_bit_xor xor_gate(.A(A), .B(B), .out(xor_out));
eight_bit_xnor xnor_gate(.A(A), .B(B), .out(xnor_out));
eight_bit_not not_gate(.A(A), .out(not_out));
eight_bit_nor nor_gate(.A(A), .B(B), .out(nor_out));

always @(*) begin

    output1 = 8'd0;
    output2 = 8'd0;

    case (opcode)
        6'd1: begin
            output1=A;
        end
        6'd3: begin
            output1=A;
        end




        6'b000100: begin // ADD
            output1 = add_sum;
            output2 = {7'd0, add_carry};
        end

        6'b000101: begin // SUB
            output1 = sub_diff;
            output2 = {7'd0, sub_borrow};
        end

        6'b000110: begin // NEG
            output1 = neg_out;
        end

        6'b000111: begin // MUL
            output1 = mul_full[7:0];
            output2 = mul_full[15:8];
        end

        6'b001000: begin // DIV
            output1 = div_quot;
            output2 = div_rem;
        end

        6'b001001: output1 = or_out;
        6'b001010: output1 = xor_out;
        6'b001011: output1 = nand_out;
        6'b001100: output1 = nor_out;
        6'b001101: output1 = xnor_out;
        6'b001110: output1 = not_out;
        6'b001111: output1 = lshift_out;
        6'b010000: output1 = rshift_out;
        default: begin
            output1=8'd0;
            output2=8'd0;
        end

    endcase

```

```
end

endmodule
```

Listing 17: alu.v

3.5 Data Memory

```
module data_memory(
    input clk,
    input data_write_enb,
    input [7:0] data_write,
    input [7:0] read_address,
    input [7:0] write_address,

    output   [7:0]data_read
);

reg [7:0] memory [255:0];

always@(posedge clk) begin
    if(data_write_enb) begin

        memory[write_address]<=data_write;
    end
end

assign data_read=memory[read_address];
endmodule
```

Listing 18: data$_memory.v$

3.6 Writeback Unit

```


`include "data_memory.v"


module write_back(

    input clk,
    input  [31:0] instruction,
    input  [7:0] output1,
    input  [7:0] output2,


    output reg [7:0] data_out1,
    output reg [7:0] data_out2,

    output reg write_enable1,
    output reg write_enable2,

    output reg [4:0] write_address1,
    output reg [4:0] write_address2,

    output reg [7:0] write_address
);
```

```

wire [5:0] opcode;
assign opcode = instruction[31:26];

reg [7:0] mem_read_address;
reg [7:0] data_write;
reg data_write_enb;

wire [7:0] mem_data_out;



always @(*) begin

    data_out1        = 8'd0;
    data_out2        = 8'd0;

    mem_read_address = 8'd0;

    write_address1   = 5'bxxxxx;
    write_address2   = 5'bxxxxx;

    write_address    = 8'bxxxxxxxx;
    data_write       = 8'd0;

    write_enable1    = 0;
    write_enable2    = 0;

    data_write_enb   = 0;

    // 000000 Immediate MOV
    if (opcode == 6'b000000) begin
        data_out1      = instruction[7:0];
        write_address1 = instruction[25:21];
        write_enable1  = 1'b1;
    end

    // 000001 Register MOV
    else if (opcode == 6'b000001) begin
        write_address1 = instruction[25:21];
        data_out1      = output1;
        write_enable1  = 1'b1;
    end

    // 000010 LOAD
    else if (opcode == 6'b000010) begin
        mem_read_address = instruction[7:0];
        write_address1   = instruction[25:21];
        data_out1        = mem_data_out;
        write_enable1    = 1'b1;
    end

    // 000011 STORE
    else if (opcode == 6'b000011) begin
    write_address = instruction[25:18]; // memory address

    data_write     = output1;           // value from pipeline
    data_write_enb = 1'b1;
end

```

```
    // 000100 ADD
    else if (opcode == 6'b000100) begin
        write_address1 = instruction[20:16];
        write_address2 = instruction[25:21];
        data_out1      = output1;
        data_out2      = output2;
        write_enable1  = 1;
        write_enable2  = 1;
    end

    // 000101 SUB
    else if (opcode == 6'b000101) begin
        write_address1 = instruction[20:16];
        write_address2 = instruction[25:21];
        data_out1      = output1;
        data_out2      = output2;
        write_enable1  = 1;
        write_enable2  = 1;
    end

    // 000110 NEG
    else if (opcode == 6'b000110) begin
        write_address1 = instruction[20:16];
        data_out1      = output1;
        write_enable1  = 1;
    end

    // 000111 MUL
    else if (opcode == 6'b000111) begin
        write_address1 = instruction[20:16];
        write_address2 = instruction[25:21];
        data_out1      = output1;
        data_out2      = output2;
        write_enable1  = 1;
        write_enable2  = 1;
    end

    // 001000 DIV
    else if (opcode == 6'b001000) begin
        write_address1 = instruction[20:16];
        write_address2 = instruction[25:21];
        data_out1      = output1;
        data_out2      = output2;
        write_enable1  = 1;
        write_enable2  = 1;
    end

    // 001001 OR
    else if (opcode == 6'b001001) begin
        write_address1 = instruction[20:16];
        data_out1      = output1;
        write_enable1  = 1;
    end

    // 001010 XOR
    else if (opcode == 6'b001010) begin
        write_address1 = instruction[20:16];
        data_out1      = output1;
        write_enable1  = 1;
    end
```

```
    // 001011 NAND
    else if (opcode == 6'b001011) begin
        write_address1 = instruction[20:16];
        data_out1      = output1;
        write_enable1  = 1;
    end

    // 001100 NOR
    else if (opcode == 6'b001100) begin
        write_address1 = instruction[20:16];
        data_out1      = output1;
        write_enable1  = 1;
    end

    // 001101 XNOR
    else if (opcode == 6'b001101) begin
        write_address1 = instruction[20:16];
        data_out1      = output1;
        write_enable1  = 1;
    end

    // 001110 NOT
    else if (opcode == 6'b001110) begin
        write_address1 = instruction[20:16];
        data_out1      = output1;
        write_enable1  = 1;
    end

    // 001111 LSHL
    else if (opcode == 6'b001111) begin
        write_address1 = instruction[20:16];
        data_out1      = output1;
        write_enable1  = 1;
    end

    // 010000 LRSH
    else if (opcode == 6'b010000) begin
        write_address1 = instruction[20:16];
        data_out1      = output1;
        write_enable1  = 1;
    end

end


// -------- DATA MEMORY CONNECTION --------

data_memory dm(

    .clk(clk),
    .data_write_enb(data_write_enb),

    .data_write(data_write),

    .read_address(mem_read_address),
    .write_address(write_address),

    .data_read(mem_data_out)

```

```
);

endmodule
```

Listing 19: writeback_unit.v

3.7 Top Processor

```

    `include "fetch_unit.v"
    `include "decode_unit.v"
    `include "write_back.v"
    `include "alu.v"
    `include "register_file2.v"



    module top(
        input clk,
        input reset,
        input write_en,
        input [31:0] instruction_in
    );



    wire [31:0] instruction_out;

    wire [7:0] reg_data_in;
    wire [7:0] write_data1;
    wire [7:0] write_data2;

    wire [7:0]  A;
    wire [7:0] B;
    wire [7:0]  output1;
    wire [7:0] output2;


    wire [4:0]  write_address1;
    wire [4:0] write_address2;
    wire [4:0]  read_address1;
    wire [4:0] read_address2;
    wire write_enable1;
    wire write_enable2;




    // ================= Register File =================

    // Read port 1
    register_file2 rf(
        .clk(clk),

        .write_enable1(write_enable1),
        .write_enable2(write_enable2),

        .write_address1(write_address1),
```

```
        .write_address2(write_address2),

        .write_data1(write_data1),
        .write_data2(write_data2),

        .read_address1(read_address1),
        .read_address2(read_address2),

        .data_out1(A),
        .data_out2(B)
    );

    // ================= Fetch =================

    fetch_unit fetch(
        .clk(clk),
        .reset(reset),
        .instruction_in(instruction_in),

        .instruction_out(instruction_out),
        .write_en(write_en)

    );

    // ================= Decode =================

    decode_unit decode(
        .instruction(instruction_out),
        .read_address1(read_address1),
        .read_address2(read_address2)
    );

    // ================= ALU =================

    alu alu_unit(
        .opcode(instruction_out[31:26]),
        .A(A),
        .B(B),
        .output1(output1),
        .output2(output2)
    );

    // ================= Write Back =================

    write_back wb(
        .clk(clk),
        .instruction(instruction_out),
        .output1(output1),
        .output2(output2),

        .write_enable1(write_enable1),
        .write_enable2(write_enable2),

        .write_address1(write_address1),
        .write_address2(write_address2),
        .data_out1(write_data1),
        .data_out2(write_data2)
    );

    endmodule
```

Listing 20: top_processor.v

4 Testbench

```

//assignment questions --(a+b+c+d)^2



`timescale 1ns/1ps
`include "top.v"

module top_tb;

reg clk;
reg reset;
reg [31:0] instruction_in;
reg write_en;

top dut(
    .clk(clk),
    .reset(reset),
    .instruction_in(instruction_in),
    .write_en(write_en)
);




initial clk = 0;
always #5 clk = ~clk;



initial begin
    $dumpfile("top.vcd");
    $dumpvars(0,top_tb);
end


// ---------------- Monitor ----------------

initial begin
$monitor("TIME=%0t | INSTR=%b | A=%d | B=%d | OUT1=%d | OUT2=%d",
$time,
dut.instruction_out,
dut.A,
dut.B,
dut.output1,
dut.output2
);
end




initial begin

reset = 1;
write_en = 1;      // enable program loading
instruction_in = 0;

```

```
#10;
reset = 0;




// MOV R1,#1
instruction_in = 32'b00000000001000000000000000000001; #10;

// MOV R2,#2
instruction_in = 32'b00000000001000000000000000000010; #10;

// MOV R3,#3
instruction_in = 32'b00000000001100000000000000000011; #10;

// MOV R4,#4
instruction_in = 32'b00000000001000000000000000000100; #10;

// ADD R5,R2,R1
instruction_in = 32'b00010000000001010000000001000001; #10;

// ADD R5,R5,R3
instruction_in = 32'b00010000000001010000000010100011; #10;

// ADD R5,R5,R4
instruction_in = 32'b00010000000001010000000010100100; #10;

// MUL R6,R5,R5
instruction_in = 32'b00011100111001100000000010100101; #10;



// MOV R7, R6
instruction_in = 32'b00000100111000000000000000000110; #10;



// STORE MEM[8],R6
instruction_in = 32'b00001100001000000000000000000110; #10;



write_en = 0;

#200;


// ---------------- Results ----------------

$display("RESULT REGISTER R7 = %d", dut.rf.registers[7]);
$display("RESULT REGISTER R6 = %d", dut.rf.registers[6]);
$display("RESULT MEMORY[8] = %d", dut.wb.dm.memory[8]);


$monitoroff;  //this is used to stop the monitor



#200;
```

```
end


integer i;
initial begin
    #1000;
    $display("=== Register File Contents ===");
    for(i = 0; i <8 ; i = i + 1) begin

        $display("registers[%0d] = %d", i, top_tb.dut.rf.registers[i]);
    end
    $finish;
end

endmodule
```

Listing 21: testbench.v

5 Simulation Results

5.1 Waveform

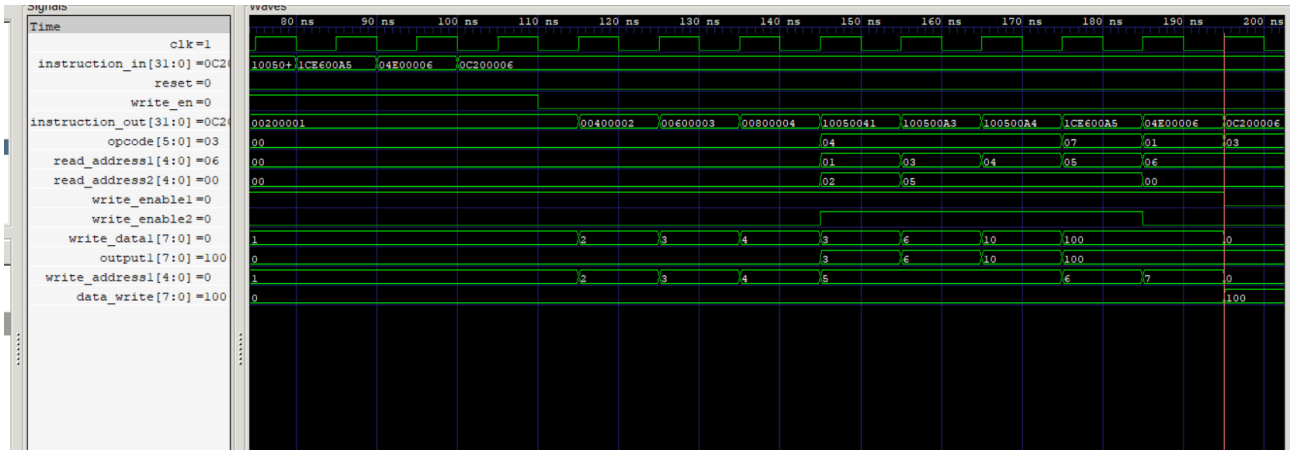

Figure 1: Simulation Waveform

5.2 Output

```
PS D:\verilog> vvp top.vvp
VCD info: dumpfile top.vcd opened for output.
TIME=0 | INSTR=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx | A=  x | B=  x | OUT1=  0 | OUT2=  0
TIME=15000 | INSTR=00000000001000000000000000000001 | A=  x | B=  x | OUT1=  0 | OUT2=  0
TIME=115000 | INSTR=00000000001000000000000000000010 | A=  x | B=  x | OUT1=  0 | OUT2=  0
TIME=125000 | INSTR=00000000001100000000000000000011 | A=  x | B=  x | OUT1=  0 | OUT2=  0
TIME=135000 | INSTR=00000000010000000000000000000100 | A=  x | B=  x | OUT1=  0 | OUT2=  0
TIME=145000 | INSTR=00010000000001010000000001000001 | A=  1 | B=  2 | OUT1=  3 | OUT2=  0
TIME=155000 | INSTR=00010000000001010000000010100011 | A=  3 | B=  3 | OUT1=  6 | OUT2=  0
TIME=165000 | INSTR=00010000000001010000000010100100 | A=  4 | B=  6 | OUT1= 10 | OUT2=  0
TIME=175000 | INSTR=00011100111001100000000010100101 | A= 10 | B= 10 | OUT1=100 | OUT2=  0
TIME=185000 | INSTR=00000100111000000000000000000110 | A=100 | B=  0 | OUT1=100 | OUT2=  0
TIME=195000 | INSTR=00001100001000000000000000000110 | A=100 | B=  0 | OUT1=100 | OUT2=  0
TIME=205000 | INSTR=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx | A=  0 | B=  0 | OUT1=  0 | OUT2=  0
RESULT REGISTER R7 = 100
RESULT REGISTER R6 = 100
RESULT MEMORY[8] = 100
=== Register File Contents ===
registers[0] =   0
registers[1] =   1
registers[2] =   2
registers[3] =   3
registers[4] =   4
registers[5] =  10
registers[6] = 100
registers[7] = 100
top_tb.v:132: $finish called at 1000000 (1ps)
PS D:\verilog>
```

Figure 2: Simulation Output

6 Logic Synthesis

6.1 Details

- Tool: Cadence Genus
- Technology: 45nm CMOS
- Generated Files:
  - netlist.v
  - constraints.sdc

6.2 Reports

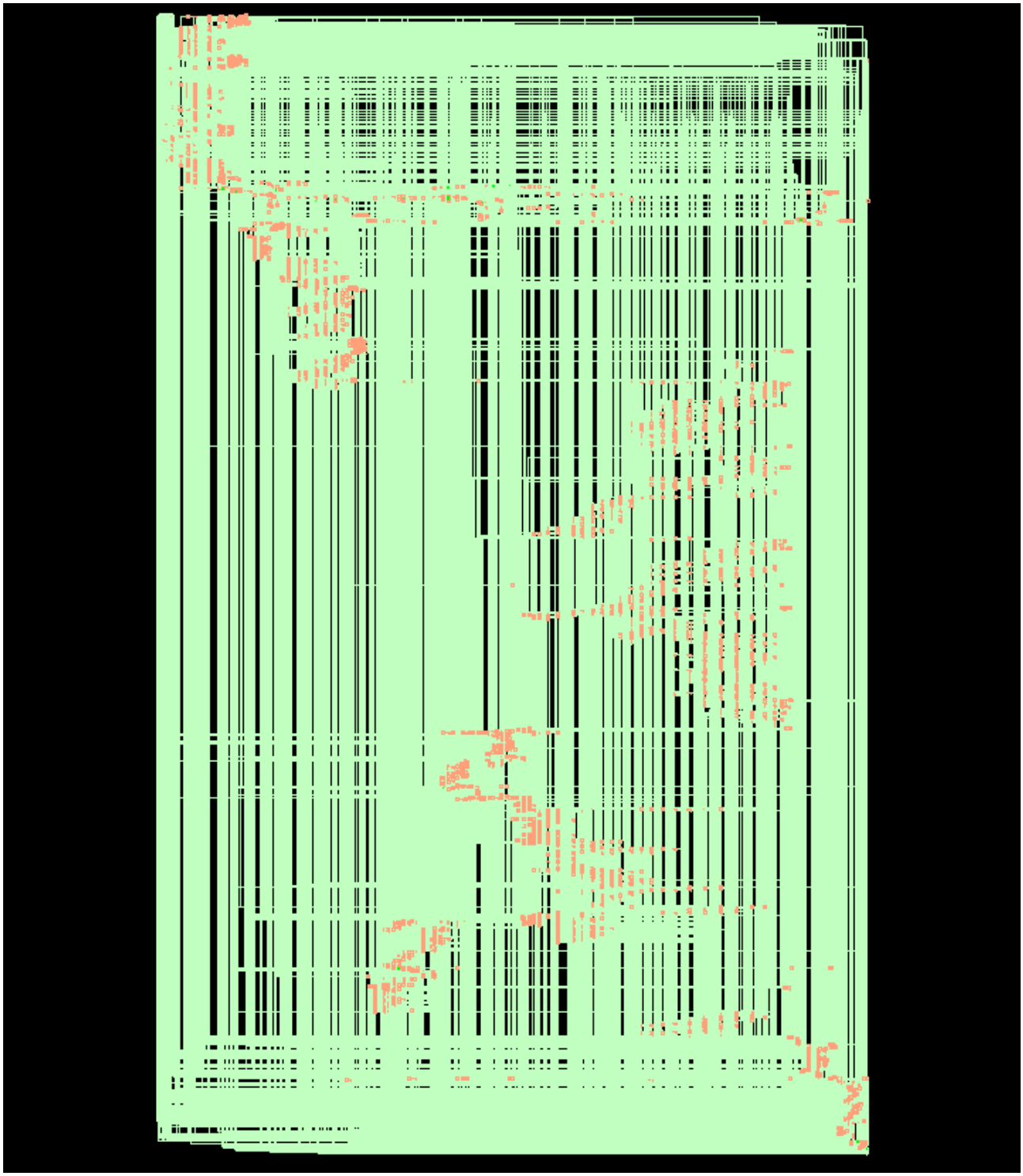

Figure 3: Schematic

Timing Report - (id: 1) — vlsi-3.iiitk.ac.in

	Airlines	Endpoint	Slack(ps)	Rise Slew(ps)	Fall Slew(ps)
1	cpath_1_1	rf_registers_reg[31][7]/D	8538	51.0	56.3

	Pin	Type	Fanout	Load(fF)	slew(ps)	Delay(ps)	Arrival(ps)	Edge
40	g109233__1666/B					0	890	
41	g109233__1666/Y	XNOR2X1	2	0.8	10	32	922	F
42	g109204__4319/AN					0	922	
43	g109204__4319/Y	NAND2BX1	2	0.6	16	24	946	F
44	g109184__5122/A					0	946	
45	g109184__5122/Y	NOR2XL	2	0.6	20	20	966	R
46	g109179/A					0	966	
47	g109179/Y	INVXL	1	0.2	6	11	977	F
48	g109178__6783/B0					0	977	
49	g109178__6783/Y	OAI21XL	2	0.6	25	10	987	R
50	g109175__4319/B					0	987	
51	g109175__4319/Y	AND2XL	2	0.8	12	27	1014	R
52	g109168__1666/A1					0	1014	
53	g109168__1666/Y	OAI32X1	1	0.4	27	26	1040	F
54	g109165__9945/C0					0	1040	
55	g109165__9945/Y	AOI221X1	1	0.5	34	31	1071	R
56	g109162__4733/C0					0	1071	
57	g109162__4733/Y	OAI221X1	1	0.2	32	36	1108	F
58	g109159__1881/C0					0	1108	
59	g109159__1881/Y	AOI211XL	1	0.3	30	29	1137	R
60	g109158__6131/D					0	1137	
61	g109158__6131/Y	NAND4XL	3	0.7	50	40	1177	F
62	g106678/B0					0	1177	
63	g106678/Y	AOI222X1	32	10.0	219	155	1331	R
64	g104025/B0					0	1331	
65	g104025/Y	OAI222XL	1	0.4	56	109	1440	F
66	rf_registers_reg[31][7]/D	DFFHQX1				0	1440	
67	rf_registers_reg[31][7]/CK	setup			0	22	1462	R
68	/designs/top/timing/clock_domains/domain_1/clk	capture					10000.0	R

Figure 4: Timing Report

Generated by: Genus(TM) Synthesis Solution 20.11-s111_1 (Apr 26 2021 14:57:38)
Generated on: Apr 05 2026 17:18:03
Module: design:top
Technology library: fast_vdd1v0 1.0
Operating conditions: PVT_1P1V_0C (balanced_tree)
Wireload mode: enclosed

Instance	Cells	Leakage (nW)	Internal (nW)	Net (nW)	Switching (nW)
top/al...e_2_c1	0	0.000	0.000	204.714	204.714
top/al...e_3_c1	0	0.000	0.000	194.884	194.884
top/al...e_4_c1	0	0.000	0.000	143.241	143.241
top/al...e_5_c1	0	0.000	0.000	153.969	153.969
top/al...e_6_c1	0	0.000	0.000	193.992	193.992
top/al...e_7_c1	0	0.000	0.000	238.155	238.155
top/al...sub_m1	0	0.000	0.000	231.258	231.258
top/al...sub_m2	0	0.000	0.000	460.507	460.507
top/al...sub_m3	0	0.000	0.000	377.630	377.630
top/al...sub_m4	0	0.000	0.000	326.527	326.527
top/al...sub_m5	0	0.000	0.000	356.362	356.362
top/al...sub_m6	0	0.000	0.000	423.983	423.983
top/al...sub_m7	0	0.000	0.000	429.574	429.574
top/al...sub_m8	0	0.000	0.000	1.089	1.089
top/al...sub_m9	0	0.000	0.000	2.904	2.904
top/al...ub_m10	0	0.000	0.000	0.000	0.000
top/al...ub_m11	0	0.000	0.000	0.000	0.000
top/al...ub_m12	0	0.000	0.000	0.000	0.000
top/al...ub_m13	0	0.000	0.000	0.000	0.000
top/al...ub_m14	0	0.000	0.000	0.363	0.363
top/al...ub_m15	0	0.000	0.000	2.178	2.178
top/al...ub_m16	0	0.000	0.000	1.452	1.452
top/al...ub_m17	0	0.000	0.000	2.178	2.178
top/al...b_m111	0	0.000	0.000	197.100	197.100
top/al...it_sum	0	0.000	0.000	1498.106	1498.106
top	8893	2021.565	1165232.915	56359.945	1221592.860

Figure 5: Power Report

Instance	Cells	Cell Area	Net Area	Total Area	Wireload	WL_Flag
top/al...ub_m16	0	0.000	0.000	0.000		(S)
top/al...ub_m17	0	0.000	0.000	0.000		(S)
top/al...b_m111	0	0.000	0.000	0.000		(S)
top/al...it_sum	0	0.000	0.000	0.000		(S)
top	8893	38309.130	0.000	38309.130		(S)

Figure 6: Gate and Area Report

7 Physical Design

- Tool: Cadence Innovus
- Aspect Ratio: 1
- Core Utilization: 0.75
- Core to Die Spacing: 0.25 μm

7.1 Floorplan

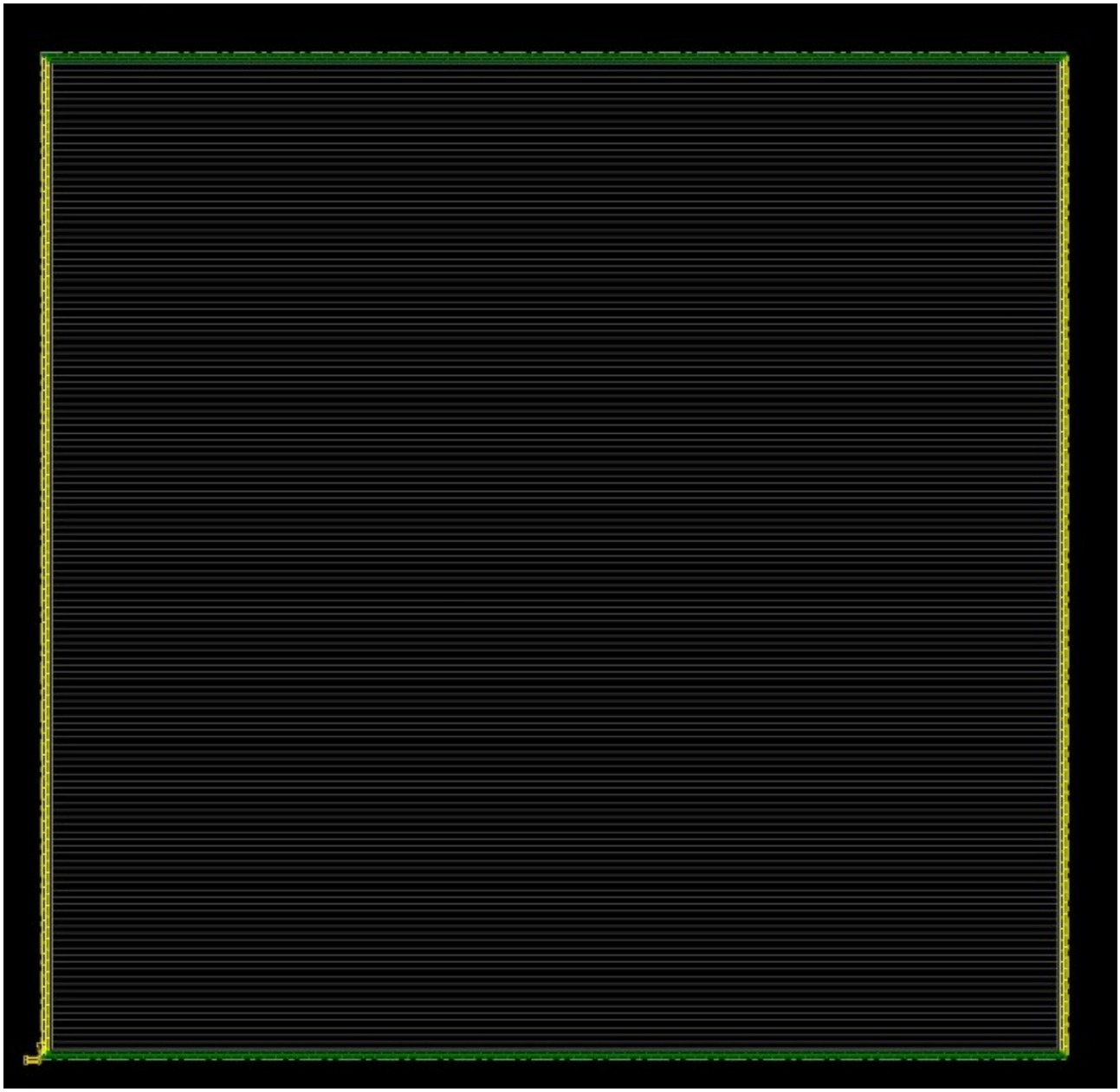

Figure 7: Floorplan showing core area and standard cell rows before placement

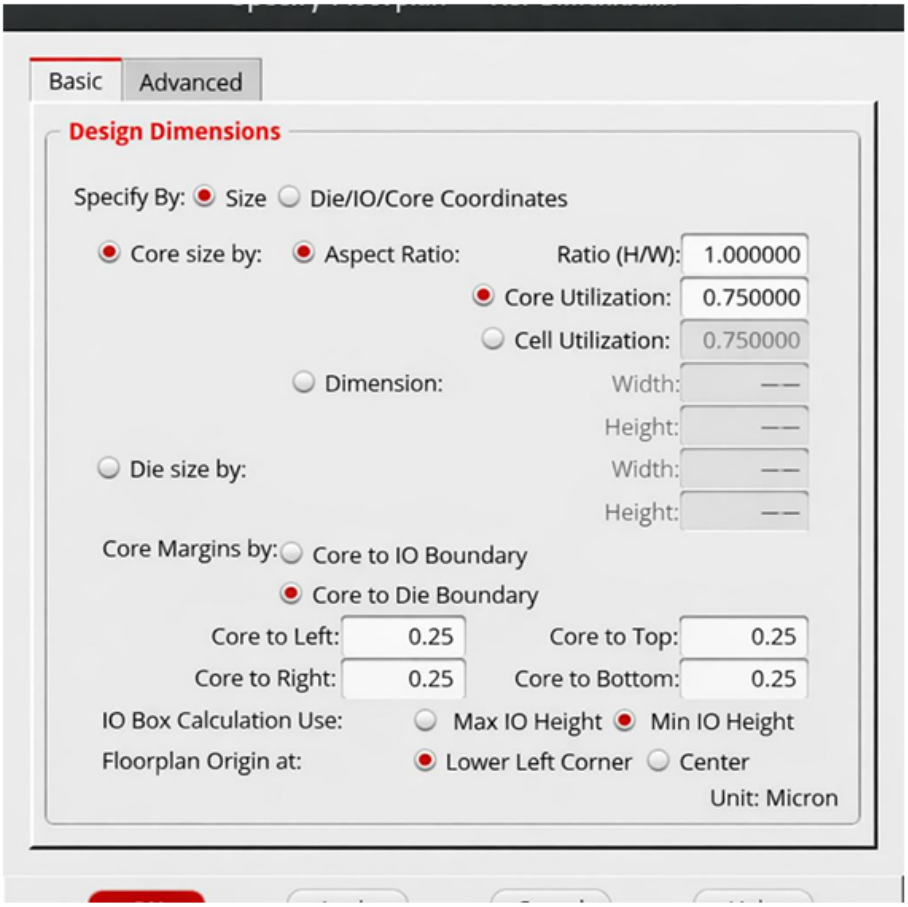

Fig – Floorplan settings showing aspect ratio, core utilization, and core-to-die spacing

Figure 8: Floorplan settings showing aspect ratio, core utilization, and core-to-die spacing

7.2 Power Plan

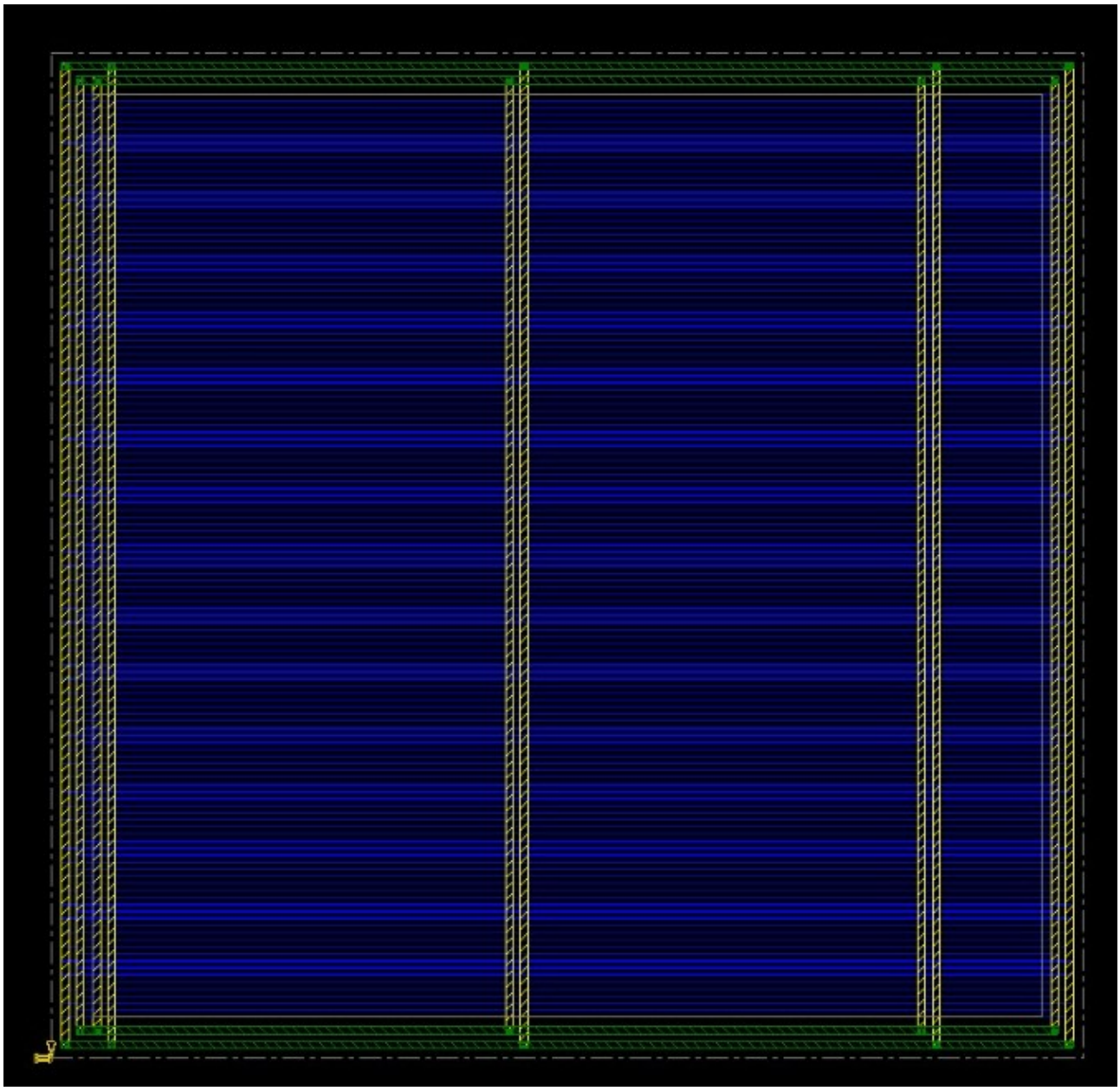

Figure 9: Power distribution network (PDN) showing VDD/VSS rings with horizontal and vertical stripes forming a mesh structure

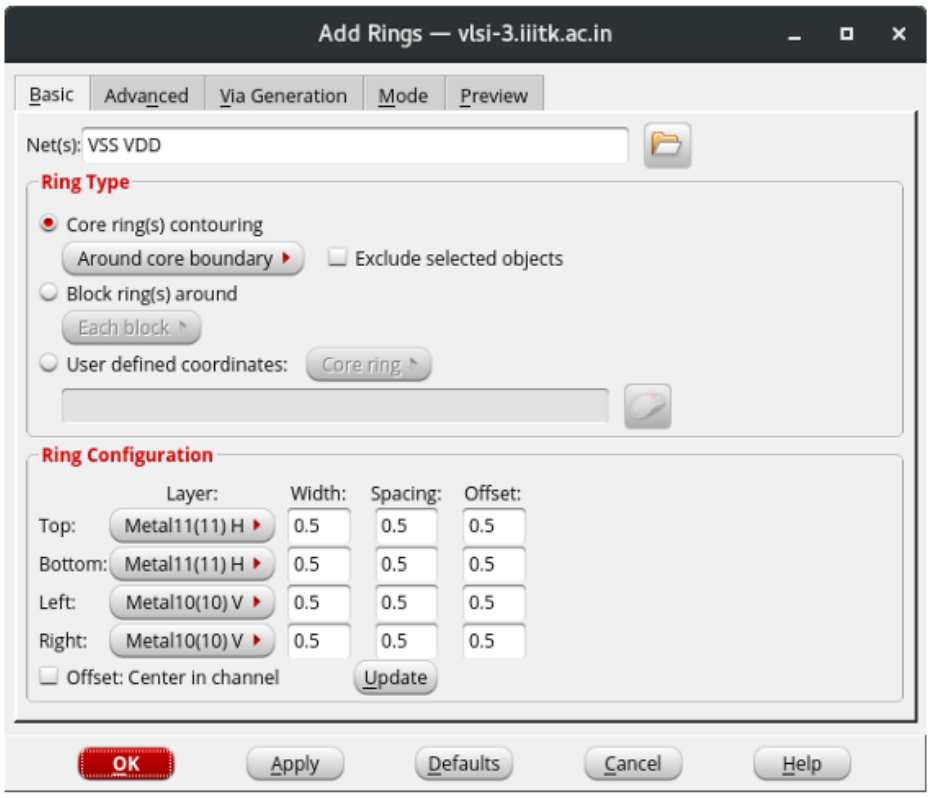

Figure 10: Ring power plan configuration showing VDD/VSS core rings around the design boundary

Figure 11: Stripe power plan configuration showing horizontal and vertical power stripes for uniform power distribution

7.3 Placement

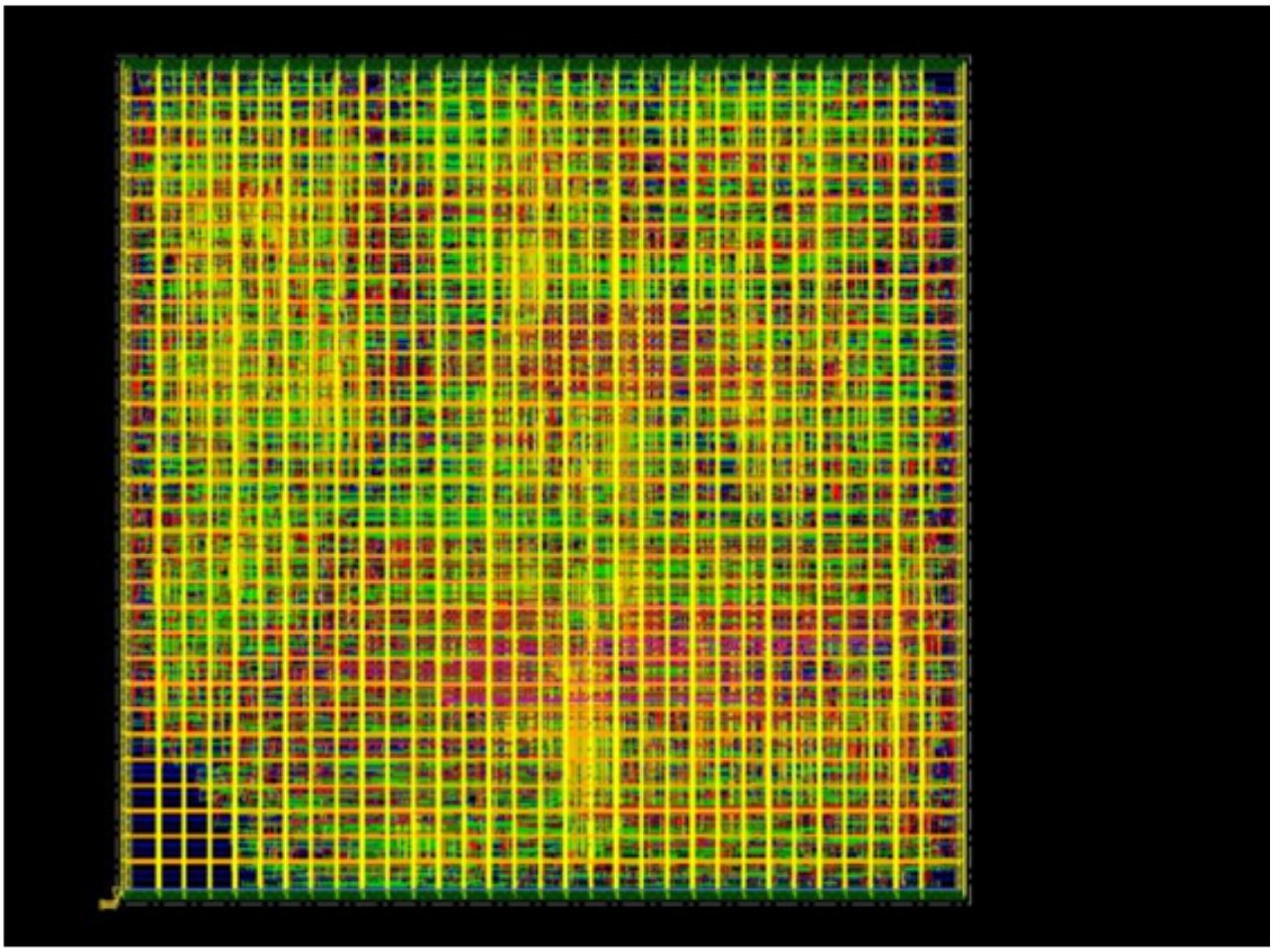

Figure 12: Placement

7.4 Routing

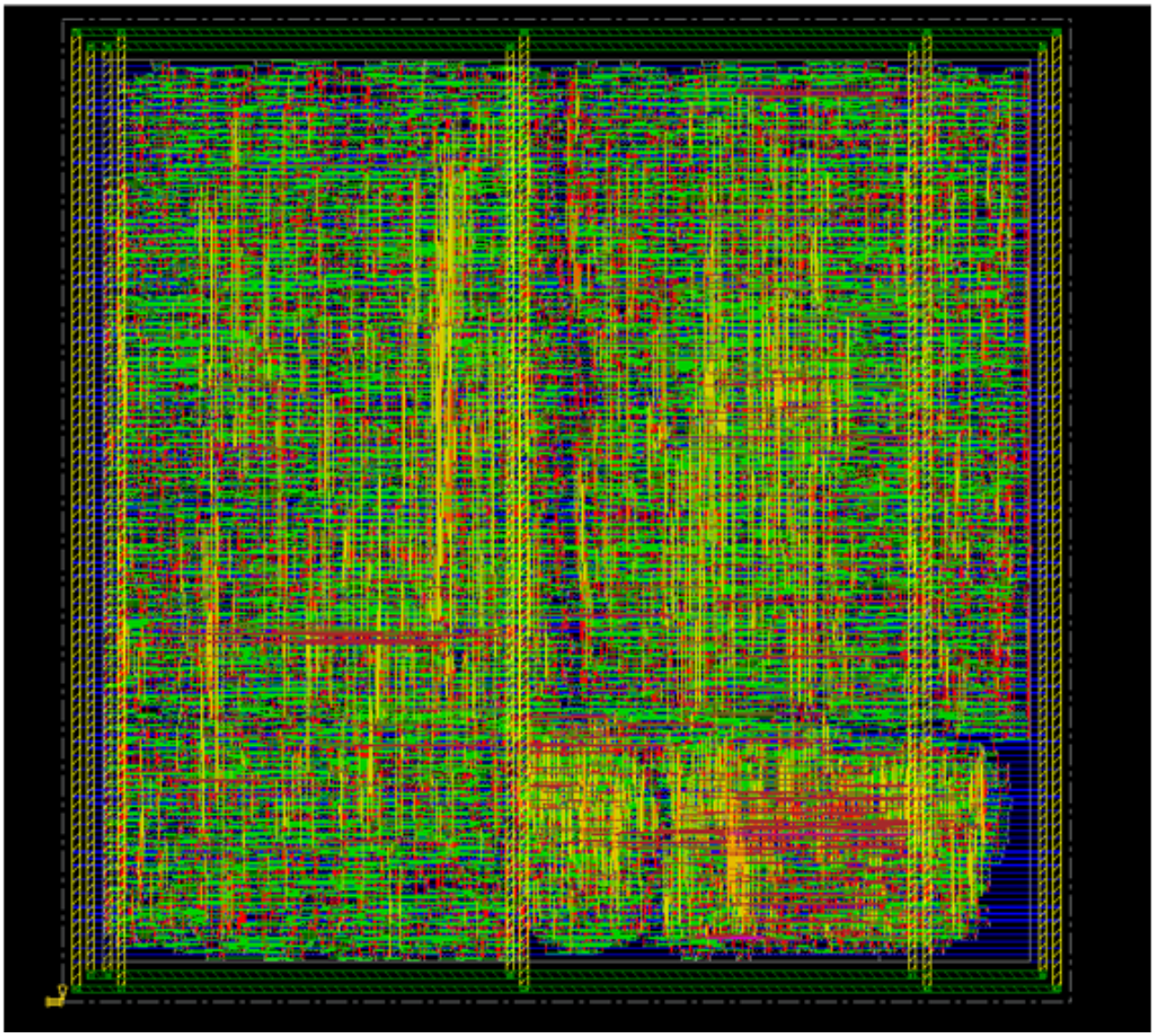

Figure 13: Routing

7.5 Final Layout

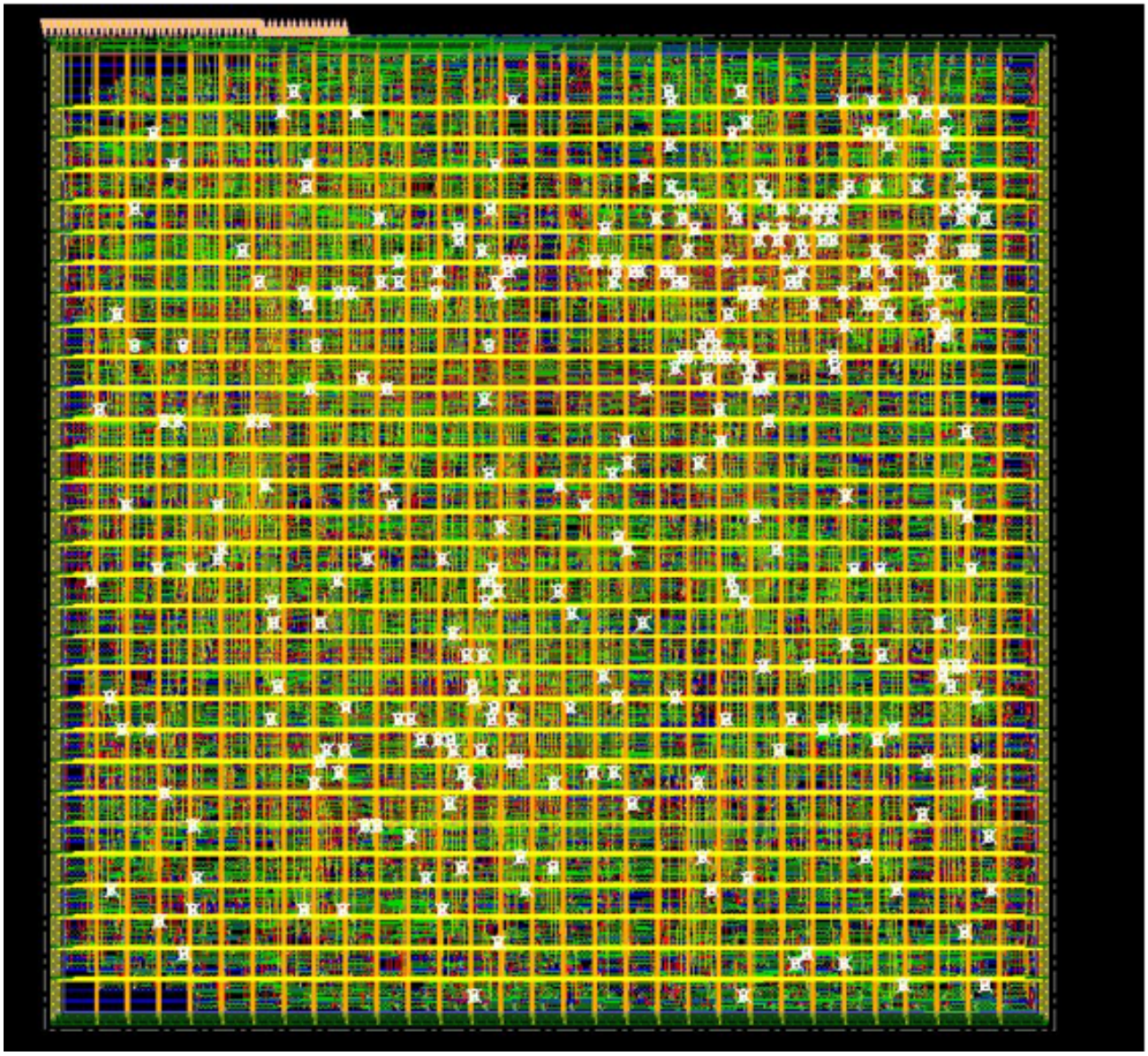

Figure 14: Final Chip Layout

8 Conclusion

- The 8-bit Harvard architecture-based processor was successfully designed and implemented using Verilog HDL.
- The processor correctly executed the fixed-point (unsigned) arithmetic expression $(a + b + c + d)^2$ using sequential ADD and MUL operations.
- Simulation results verified the functional correctness of instruction execution, datapath flow, and register operations.
- Logic synthesis using 45nm CMOS technology in Cadence Genus and physical design using Cadence Innovus were successfully completed.
- Key performance metrics such as area, power, and timing were analyzed, demonstrating efficient processor design.